

ZSH Manual Indentation Parser - Design & Implementation

Overview

A specialized indentation-based parsing system for the ZSH manual, building on existing indent tools from the ZshTools collection. This system creates a hierarchical navigation structure for man pages loaded into arrays.

Design Principles

1. **Pure ZSH where practical** - Minimize external dependencies
2. **Reuse existing patterns** - Leverage auto-EOF reader, tree structures, etc.
3. **Composable functions** - Each function does one thing well
4. **Hierarchical navigation** - Man pages have natural tree structure
5. **Performance conscious** - Handle large man pages efficiently

Architecture



Core Components

1. Enhanced Control Sequence Stripper

zsh

```
#!/usr/bin/zsh
```

```
# Enhanced version using collection patterns
```

```
function @man:stripCtrlSeqs {
```

```
    ## Removes all control sequences and formatting from man page text *##
```

```
    local AwkStripCtrlSeqs='
```

```
{
```

```
    # ANSI escape sequences
```

```
    gsub(/\x1b\[[0-9;]*[mGKHFJ]/, "")
```

```
    gsub(/\x1b\.[\x07/, "")    # OSC sequences
```

```
    gsub(/\x1b[>=]/, "")    # Other escapes
```

```
    # Man page specific formatting
```

```
    gsub(/.\x08/, "")    # Backspace overstrikes
```

```
    gsub(/\x0d/, "")    # Carriage returns
```

```
    # Control characters except tab and newline
```

```
    gsub(/[\x00-\x08\x0B-\x1F\x7F]/, "")
```

```
    # Original patterns
```

```
    gsub(/^\[[0-9;]+m|M\-\^\@M\-\^P|M\-b/, "")
```

```
    print $0
```

```
}
```

```
,
```

```
# Use auto-EOF pattern with no blank line waiting
```

```
local MAX_BLANK=0
```

```
local LINE CNT=0
```

```
while [[ ${${LINE}:= "$(IFS= read -rse)"; +${( CNT=0 )}}; -${(CNT++)} -lt ${MAX_BLANK} ]] {
```

```
    print -r -- "$LINE" | awk "$AwkStripCtrlSeqs"
```

```
}
```

```
}
```

2. Indent Measurement with Metadata

```
function @man:measureIndent {
    ## Measures indentation and adds metadata for man page structure ##

    local AwkMeasureIndent='
BEGIN {
    SectionPattern = "^[:space:]*[A-Z][A-Z]+[:space:]*$"
    SubsectionPattern = "^[:space:]+[A-Z][a-z]"
    BulletPattern = "^[:space:]+[•◦◐◑◒◓◔◕◖◗◘◙◚◛◜◝◞◟◠◡◢◣◤◥◦◧◨◩◪◫◬◭◮◯◰◱◲◳◴◵◶◷◸◹◺◻◼◽◾◿◸◹◺◻◼◽◾◿"

}

/^[:space:]*$/ {
    print "BLANK", 0, 0, $0
    next
}

{
    match($0, /^[:space:]+/, MATCH)
    indent = RLENGTH == -1 ? 0 : RLENGTH

    # Detect line type
    type = "TEXT"
    if ($0 ~ SectionPattern) type = "SECTION"
    else if ($0 ~ SubsectionPattern) type = "SUBSECTION"
    else if ($0 ~ BulletPattern) type = "BULLET"

    print type, indent, length($0), $0
}
'
```

zsh

```
function @man:normalizeIndentLevels {  
    ## Normalizes indent levels and preserves document order ##  
  
    local AwkNormalize=  
    BEGIN {  
        LastIndent = -1  
        Level = 0  
        delete LevelMap  
    }  
  
    {  
        Type = $1  
        Indent = $2  
        Length = $3  
        $1 = $2 = $3 = ""  
        Line = substr($0, 4) # Remove leading spaces from field removal  
  
        # Skip blank lines in level calculation  
        if (Type == "BLANK") {  
            print 0, Type, Indent, Length, Line  
            next  
        }  
  
        # Determine hierarchical level  
        if (! (Indent in LevelMap)) {  
            if (Indent > LastIndent) {  
                Level++  
            } else if (Indent < LastIndent) {  
                # Find appropriate level  
                for (l in LevelMap) {  
                    if (LevelMap[l] == Indent) {  
                        Level = l  
                        break  
                    }  
                }  
            }  
            if (! (Indent in LevelMap)) {  
                Level = 1  
                for (l in LevelMap) {  
                    if (LevelMap[l] < Indent) Level++  
                }  
            }  
        }  
        LevelMap[Level] = Indent  
        LastIndent = Indent  
    } else {
```

```

    } else {
        # Find level for known indent
        for (l in LevelMap) {
            if (LevelMap[l] == Indent) {
                Level = l
                break
            }
        }
    }

    print Level, Type, Indent, Length, Line
}
'

awk "$AwkNormalize"
}

```

4. Tree Structure Builder

zsh

```
function @man:buildTree {
    local TreeName="${1:-ManTree}"
    local -a Input=("${@:2}") # Array of lines if provided

    ## Build nested tree structure from indented text ##

    # Initialize tree
    @tree:new $TreeName

    # State tracking
    local -a NodeStack=($TreeName)
    local -i CurrentLevel=0
    local -i LineNum=0

    # Section tracking for special handling
    local CurrentSection=""
    local CurrentSubsection=""

    # Process input
    local InputCmd="print -l \ $Input"
    [[ $#Input } -eq 0 ]] && InputCmd="@man:measureIndent | @man:normalizeIndentLevels"

    eval "$InputCmd" | while read -r Level Type Indent Length Line; do
        (( LineNum++ ))

        # Skip blank lines
        [[ $Type == "BLANK" ]] && continue

        # Sanitize node name (replace spaces and special chars)
        local NodeName="${Line//[ ^a-zA-Z0-9_ ]/_}"
        NodeName="${NodeName:0:50}" # Limit length

        # Adjust stack based on level
        if (( Level > CurrentLevel )); then
            # Going deeper - current line is child of previous
            NodeStack+=("${NodeStack[-1]}.${NodeName}")
        elif (( Level < CurrentLevel )); then
            # Going up - pop stack
            local -i PopCount=$((CurrentLevel - Level + 1))
            repeat $PopCount { NodeStack[-1]=() }
            NodeStack[-1]="${NodeStack[-2]}.${NodeName}"
        else
            # Same level - replace last element
            NodeStack[-1]="${NodeStack[-2]}.${NodeName}"
        fi
    done
}
```

fi

Store node data

eval "\${NodeStack[-1]}:line=\"\\${LineNum}\""

eval "\${NodeStack[-1]}:type=\"\\${Type}\""

eval "\${NodeStack[-1]}:indent=\"\\${Indent}\""

eval "\${NodeStack[-1]}:content=\"\\${Line}\""

Special handling for sections

if [[\$Type == "SECTION"]]; then

 CurrentSection="\$NodeName"

 eval "\${TreeName}:sections:\${CurrentSection}=\"\\${LineNum}\""

elif [[\$Type == "SUBSECTION" && -n \$CurrentSection]]; then

 CurrentSubsection="\$NodeName"

 eval "\${TreeName}:sections:\${CurrentSection}:\${CurrentSubsection}=\"\\${LineNum}\""

fi

 CurrentLevel=\$Level

done

Store metadata

eval "\${TreeName}:metadata:total_lines=\"\\${LineNum}\""

eval "\${TreeName}:metadata:parse_date=\"\\$(date +%s)\""

}

5. Navigation Interface

zsh

Quick section jumper

```
function @man:goto {
    local Section="${1:?Section name required}"
    local TreeName="${2:-ManTree}"

    # Try direct section lookup first
    local LineNum
    eval "LineNum=\${${TreeName}:sections:${Section}}"

    if [[ -z $LineNum ]]; then
        # Fuzzy match section names
        local -a Matches
        eval "Matches=( \${(k)${TreeName}:sections} )"
        Matches=( ${ (M)Matches:##*${Section}*} )

        if [[ ${#Matches} -eq 1 ]]; then
            eval "LineNum=\${${TreeName}:sections:${Matches[1]}}"
        elif [[ ${#Matches} -gt 1 ]]; then
            print "Multiple matches found:"
            print -l $Matches
            return 1
        else
            print "Section not found: $Section"
            return 1
        fi
    fi

    # Display section (with context)
    local -i StartLine=$((LineNum - 2))
    local -i EndLine=$((LineNum + 20))
    print -l $ZshMan[$StartLine,$EndLine]
}
```

List all sections

```
function @man:sections {
    local TreeName="${1:-ManTree}"

    print "Available sections:"
    eval "print -l \${(k)${TreeName}:sections}"
}
```

Extract full section content

```
function @man:extract {
    local Section="${1:?Section name required}"
    local TreeName="${2:-ManTree}"
```



```
local TreeName=${2:-man1tree}
```

```
# Get section start line
```

```
local -i StartLine
```

```
eval "StartLine=\${${TreeName}:sections:${Section}:-0}"
```

```
[[ $StartLine -eq 0 ]] && {
```

```
    print "Section not found: $Section"
```

```
    return 1
```

```
}
```

```
# Find next section
```

```
local -i EndLine=${#ZshMan}
```

```
local -a AllSections
```

```
eval "AllSections=( \${(v)${TreeName}:sections} )" "
```

```
for Line in ${(n)AllSections}; do
```

```
    if (( Line > StartLine && Line < EndLine )); then
```

```
        EndLine=$((Line - 1))
```

```
    fi
```

```
done
```

```
print -l $ZshMan[$StartLine,$EndLine]
```

```
}
```

```
# Search within manual with context
```

```
function @man:search {
```

```
    local Pattern="${1:?Search pattern required}"
```

```
    local Context="${2:-3}" # Lines of context
```

```
    local -a Matches
```

```
    local -i LineNum=0
```

```
print -l $ZshMan | while read -r Line; do
```

```
    (( LineNum++ ))
```

```
    if [[ $Line =~ $Pattern ]]; then
```

```
        local -i Start=$((LineNum - Context))
```

```
        local -i End=$((LineNum + Context))
```

```
        print "\n--- Match at line $LineNum ---"
```

```
        print -l $ZshMan[$Start,$End] | @man:highlightPattern "$Pattern"
```

```
    fi
```

```
done
```

```
}
```

```
# Highlight pattern in output (using your collection pattern)
```

```
function @man:highlightPattern {
```

```
    local Pattern="${1}"
```

```
awk -v pattern="$Pattern" '
{
    gsub(pattern, "\\033[1;33m&\\033[0m")
    print
}'
}
```

6. Usage Examples

zsh

Load manual into array

```
declare -ga ZshMan=( "${@f$(man zshall)}" )
```

Build navigation tree

```
@man:buildTree ManTree
```

Navigate to sections

```
@man:sections          # List all sections
```

```
@man:goto DESCRIPTION  # Jump to DESCRIPTION
```

```
@man:goto "PARAMETER EXPANSION" # Jump to specific section
```

```
@man:extract SYNOPSIS   # Extract full SYNOPSIS section
```

Search with highlighting

```
@man:search "substitution"  # Find all mentions
```

```
@man:search "^[:space:;]+The" # Find indented sentences
```

Direct tree access

```
echo ${ManTree:sections:DESCRIPTION} # Get line number
```

```
echo ${ManTree:sections:OPTIONS:AUTOCD} # Nested sections
```

Advanced: Build custom view

```
function @man:showSubsections {
```

```
    local Section="${1}"
```

```
    @man:extract "$Section" | @man:measureIndent | while read Type Indent Length Line; do
```

```
        [[ $Type == "SUBSECTION" ]] && print "$Line"
```

```
done
```

```
}
```

Integration Points

With @tree System

- Uses nested tree structure for hierarchical navigation
- Stores metadata at each node
- Enables path-based access to sections

With @ge Tools

- Can pipe through @ge:match for filtering
- Use @ge:sub for text transformation
- Chain with other text processing tools

With @print Tools

- Format output with @print:linebreak
- Use progress bars for long operations
- Colored output for better visibility

Performance Considerations

1. **Initial Parse:** One-time cost to build tree
2. **Memory:** Full manual in array + tree structure
3. **Access:** $O(1)$ section lookup after indexing
4. **Search:** Linear but can be optimized with indexing

Future Enhancements

1. **Persistent Cache:** Save parsed tree to disk
2. **Incremental Updates:** Refresh only changed sections
3. **Cross-References:** Build link graph between sections
4. **Export Formats:** Generate markdown, HTML, etc.
5. **Interactive Browser:** TUI for navigation
6. **Pattern Learning:** Auto-detect structure patterns

Testing

```
zsh
```

```
# Test basic parsing
```

```
print "Test line\n Indented\n  More indent" | @man:measureIndent
```

```
# Test tree building
```

```
@tree:new TestTree
```

```
print "Section\n Subsection\n  Content" | @man:buildTree TestTree
```

```
# Verify structure
```

```
@tree:traverse TestTree
```

